

Architektur von Client und Server

Server:

- Modular: TWMailerServer, AuthManager, Blacklist, MailStore, Utils
- Hauptloop mit blockierendem accept(), pro Verbindung ein eigener Thread
- Jeder Thread verarbeitet eine komplette Session (LOGIN, SEND, LIST, READ, DEL)
- Nachrichtenspeicherung im Dateisystem (<spool>/<user>/<timestamp>.msg)
- Zugriff auf MailStore durch mutex geschützt
- AuthManager & Blacklist thread-safe; parallele Authentifizierungen möglich
- Textbasiertes Protokoll:
 - LOGIN, SEND (mit dot-terminated body), LIST, READ, DEL, QUIT
 - Antworten: OK oder ERR

Client:

- Konsoleninterface mit Prompt >
- Baut TCP-Verbindung über IP/Port auf
- Passwort-Eingabe ohne Echo (termios)
- Befehlsverarbeitung über eigenständige Handler (login/send/list/...)
- Empfängt Serverantworten mit recv_line()
- Vertraut auf die klare Struktur des TWMailer-Protokolls

Verwendete Technologien & Bibliotheken

- C++17, POSIX Sockets (socket, bind, listen, accept, connect)
- Multithreading: std::thread, std::mutex, std::lock_guard
- LDAP-Authentifizierung über OpenLDAP (ldap_initialize, ldap_start_tls_s, ldap_sasl_bind_s)
- TLS für abgesicherte LDAP-Verbindungen
- std::filesystem für Mail-Spool
- std::chrono für Zeit & Timestamping
- termios für Passwortverbergen
- Fehlerbehandlung über errno, perror, und Ausnahmen

Entwicklungsstrategie & Protokollanpassungen

Refactoring-Ziele:

- Saubere Trennung der Verantwortlichkeiten (Auth, Blacklist, MailStore, Server)
- Thread-Sicherheit für Blacklist, Login-Versuche und MailStore
- Klare Command-Handler statt verschachtelter Logik
- Einheitliches Dateiformat und strukturierter MailStore
- Einfache, erweiterte Protokollkommandos

Neue bzw. angepasste Protokollelemente:

- LOGIN
- Fehlversuche führen zu temporären Blacklist-Einträgen
- SEND, LIST, READ, DEL sind erst nach LOGIN erlaubt
- SEND-Nachrichten verwenden dot-terminated body (.\n)
- Server sendet beim Verbindungsaufbau ein initiales „OK“

Fehlerbehandlung:

- Jede Serverantwort endet eindeutig mit OK oder ERR
- Ungültige Kommandos → ERR
- Falsche Nutzer/Subjects → sofortige Ablehnung
- Netzwerkfehler führen zur sauberen Session-Beendigung

SynchronisationsmethodenWarum notwendig:

- Der Server ist multithreaded, mehrere Clients arbeiten parallel
- Geschützte Ressourcen:
 - Blacklist-Datei und ihre Map
 - Login-Versuchs-Tracking (attempts_)
 - Dateioperationen im MailStore

Eingesetzte Mechanismen:

- std::mutex + lock_guard schützen:
 - Blacklist-Zugriffe und Cleanup
 - Eintragungen bei Fehlversuchen
 - MailStore-Operationen (Write/Delete/List/Read)
- Jeder Thread besitzt einen eigenen Socket, daher keine Überschneidungen
- Blacklist wird automatisch mittels garbage_collect() von abgelaufenen Banns bereinigt

Handling großer Nachrichten

- Nachrichtentexte werden zeilenweise verarbeitet, nicht als kompletter Block → kein großer RAM-Verbrauch.
- Der Body wird mittels dot-terminated protocol (.\n) übertragen, sodass keine Größe vorab bekannt sein muss.
- Der Server schreibt den Text direkt in die Maildatei, ohne Zwischenspeicherung; die maximale Größe wird praktisch nur durch das Dateisystem begrenzt.

- `recv_line()` schützt vor Missbrauch durch ein Maximal-Längenlimit pro Zeile, verhindert damit Überläufe oder Denial-of-Service.